

Projet Mécatronique : NAVI N.O.T

Cahier de conception



Equipe :

Théophile de Poncins

Oscar Chalopin

Nicolas Hennebert

Version du : 18 novembre 2025

SOMMAIRE

1) Introduction.....	3
2) Description générale.....	3
3) Architecture et conception.....	4
4) Détails techniques.....	10
5) Planification et phases de développement.....	14
6) Annexe.....	15

1) Introduction

Dans le cadre de la production aéronautique, les opérateurs transportent manuellement des petites séries de pièces (jusqu'à 3 kg) entre postes de travail, ce qui génère des pertes de temps et des risques d'erreur de comptage.

L'objectif est de concevoir et réaliser un **AGV miniature** capable de :

- Transporter automatiquement 3 kg de pièces sur 20 m
- Mesurer et compter les pièces à l'unité près
- Communiquer avec le système d'information usine (ERP/MES)
- Être développé avec ROS (contrainte pédagogique)

Portée du document

Ce cahier décrit l'ensemble de la conception fonctionnelle, logicielle et matérielle du prototype final. Il exclut la conception industrielle (série, certification) et la gestion de parc.

2) Description générale

Le projet NAVI N.O.T (Navigation Optimisée de Transport) consiste à concevoir et réaliser un AGV miniature autonome de dimensions 40 × 25 × 20 cm, capable de transporter une charge utile maximale de 3 kg de pièces aéronautiques sur une distance minimale de 20 mètres. Le robot intègre une balance embarquée permettant de mesurer la masse transportée avec une précision de $\pm 0,1$ g et de déterminer automatiquement le nombre de pièces à l'unité près.

Il fonctionne sous ROS Noetic sur un Raspberry Pi5, utilise un LiDAR RPLIDAR A1 pour la navigation dans un environnement cartographié, et communique en temps réel avec le système de gestion de l'usine (ERP/MES) via Wi-Fi. Quatre roues Mecanum assurent une mobilité omnidirectionnelle, tandis qu'une interface locale (écran LCD 16×2 et LEDs) affiche les paramètres essentiels et les alertes.

Le système gère automatiquement le retour au poste de charge, les ajustements de vitesse selon la charge, ainsi que les exceptions (perte de réseau, panne capteur, etc.).

Public cible et utilisateurs :

Le principal utilisateur est l'opérateur de production, qui place les pièces sur la plateforme, vérifie le comptage affiché et reçoit les alertes visuelles. Le responsable ordonnancement consulte les données transmises dans l'ERP pour assurer la traçabilité des lots. Enfin, le technicien de maintenance bénéficie d'un accès rapide aux composants et d'un affichage clair des codes d'erreur pour un diagnostic efficace. Le robot est conçu pour être intuitif : aucune formation spécifique n'est requise pour l'opérateur.

Contraintes et hypothèses

Le projet respecte un budget matériel limité à 1200 € et l'utilisation obligatoire de ROS Noetic (contrainte pédagogique). Les contraintes techniques incluent une charge utile maximale de 3 kg, une autonomie minimale d'1 heure en fonctionnement chargé, et une vitesse cible de 0,5 m/s (réduite à 0,3 m/s avec charge). L'environnement d'utilisation est une usine aéronautique avec un sol lisse, une température comprise entre 15 et 30 °C et des vibrations modérées.

3) Architecture et conception

Architecture du système

L'architecture du NAVI N.O.T repose sur une architecture distribuée à deux niveaux de contrôle, combinant un ordinateur haute performance (Raspberry Pi 5) pour les tâches cognitives et un microcontrôleur temps réel pour les opérations bas niveau. Cette séparation garantit à la fois la puissance de traitement nécessaire à la navigation autonome et la réactivité indispensable au contrôle moteur.

Niveau supérieur - Raspberry Pi 5 : Le Raspberry Pi 5 constitue le cerveau décisionnel du système. Il héberge l'environnement ROS Noetic et assure le traitement des données du capteur LiDAR EAI YDLIDAR pour la localisation (SLAM) et la planification de trajectoire. Ce ordinateur gère également la cartographie de l'environnement pré-enregistrée, calcule les consignes de vitesse pour chaque roue Mecanum selon la cinématique omnidirectionnelle, et pilote l'interface de communication Wi-Fi avec le système d'information de l'usine (ERP/MES). Le boîtier ventilé assure une dissipation thermique efficace, évitant tout thermal throttling lors des phases de calcul intensif, notamment pendant l'évitement d'obstacles dynamiques ou la fusion de données

multi-capteurs.

Niveau inférieur - Arduino : L'Arduino assume le rôle de contrôleur temps réel. Il reçoit les consignes de vitesse depuis la Raspberry Pi et les traduit en signaux PWM modulés pour les deux ponts H L298N. Chaque pont H pilote deux moteurs CC 12V, permettant un contrôle indépendant des quatre roues Mecanum. L'Arduino implémente quatre boucles PID dédiées (une par moteur) pour asservir précisément la vitesse de rotation mesurée par les encodeurs intégrés aux motoréducteurs RS PRO. Il assure également l'acquisition temps réel des capteurs de sécurité (arrêt d'urgence, télémètres de proximité) et des cellules de charge pour la mesure de masse embarquée. En cas de détection d'obstacle critique (<20 cm) ou d'activation du bouton d'arrêt d'urgence, l'Arduino coupe instantanément l'alimentation moteur sans attendre l'accord de la Raspberry Pi, garantissant un temps de réaction <100 ms conforme à la norme NF EN 1525.

Sous-systèmes auxiliaires : Le système de mesure de masse repose sur quatre cellules de charge (capacité 5 kg chacune) disposées aux quatre coins de la plateforme, connectées à un amplificateur HX711 interfacé avec l'Arduino. Cette configuration permet une précision de ± 2 g après calibration. L'interface utilisateur locale combine un afficheur LCD 16x2 caractères (affichage du poids, batterie, destination, codes d'erreur) et quatre LEDs de signalisation (vert/rouge, fixe/clignotant) pilotées directement par l'Arduino. Le système énergétique utilise une batterie LiPo 3S 11,1V 5000mAh surveillée par un capteur de tension analogique, permettant l'estimation de l'autonomie restante et le déclenchement du protocole de retour automatique à la charge lorsque le seuil de 20% est franchi.

Modèles de données

Structure de données ROS

Le système échange des informations entre nœuds ROS selon les topics suivants :

- `/cmd_vel` (geometry_msgs/Twist) : Consignes de vitesse linéaire (v_x , v_y) et angulaire (ω_z) calculées par le planificateur de trajectoire, publiées vers l'Arduino via le nœud `serial_bridge`.
- `/scan` (sensor_msgs/LaserScan) : Nuage de points 2D du LiDAR (360°, résolution angulaire 1°, portée 0,15-6 m), publié pour la détection d'obstacles et la localisation SLAM.

- /odom (nav_msgs/Odometry) : Estimation de la pose (x, y, θ) et des vitesses du robot fusionnant odométrie des roues et données du LiDAR, calculée par le nœud *robot_localization*.
- /load_weight : Masse mesurée en kilogrammes.
- /battery_status (sensor_msgs/BatteryState) : Tension, pourcentage estimé, courant de décharge, publié pour la supervision énergétique et le déclenchement des alertes.
- /emergency_stop (std_msgs/Bool) : Signal booléen d'arrêt d'urgence, priorité absolue sur tous les autres topics.

Structure de communication ERP

Les échanges avec le système d'information usine suivent un protocole REST via Wi-Fi:

- POST /api/agv/mission : Réception d'une nouvelle mission.
- PUT /api/agv/status : Transmission périodique (toutes les 5 s) de l'état du robot (ID AGV, position GPS locale, charge transportée en kg, nombre de pièces estimé, niveau batterie %, état opérationnel).
- GET /api/agv/map : Téléchargement de la carte de navigation mise à jour (format PGM + métadonnées YAML) en cas de modification de l'environnement usine.

Calcul du nombre de pièces

Le nombre de pièces transportées N est calculé selon l'algorithme suivant après stabilisation de la mesure de masse :

1. Acquisition de la tare : Mesure de la masse plateforme vide M_{vide} lors de la phase d'initialisation (moyenne sur 100 échantillons).
2. Mesure de la charge : Acquisition de la masse totale M_{totale} après dépôt des pièces.
3. Calcul de la masse nette : $M_{nette} = M_{totale} - M_{vide}$.
4. Division par la masse unitaire : $N = \text{round}(M_{nette} / M_{unitaire})$, où $M_{unitaire}$ est la masse d'une pièce paramétrée dans le système (ex : 15 g pour une vis aéronautique spécifique).
5. Validation de cohérence : Si le résidu ($M_{nette} \bmod M_{unitaire}$) $> 3 \text{ g}$, une alerte "E03 - Anomalie comptage" est levée, suggérant la présence d'une pièce défectueuse ou d'un corps étranger.

Design des interfaces

Boutons physiques : Deux boutons sont intégrés sur le panneau frontal :

- Arrêt d'urgence (rouge) : Coupure mécanique de l'alimentation moteur via relais électromagnétique.
- Interrupteur général (à bascule, 2 positions) : Activation/désactivation complète du système, commande le relais maître 12V alimentant tous les sous-systèmes.

Signalisation LED : Quatre LEDs (2 vertes, 2 rouges) pilotées par l'Arduino selon le code couleur spécifié au cahier des charges :

- LED1 (Verte fixe) : Système opérationnel en fonctionnement nominal.
- LED2 (Verte clignotante) : Batterie faible (<20%), retour à la charge imminent.
- LED3 (Rouge fixe) : Erreur système nécessitant intervention (E01-E05).
- LED4 (Rouge clignotante) : Obstacle détecté, re-calcule de trajectoire en cours.

Règles ou normes appliquées, choix technologiques

Conformité normative

Le système respecte les normes industrielles suivantes :

NF EN ISO 12100 (Sécurité des machines) : Intégration de barrières de sécurité à trois niveaux : détection LiDAR avec zone d'arrêt à 20 cm, arrêt d'urgence matériel redondant, et signalisation visuelle claire.

NF EN 1525 (Véhicules sans conducteur) : Distance d'arrêt calculée à 8 cm pour 0,5 m/s avec 3 kg, conforme à la limite de 20 cm. Redondance des capteurs (LiDAR + ultrasons) garantissant un taux de détection de 99,9%.

NF EN 60204-1 (Équipements électriques) : Isolation galvanique entre circuits de puissance (12V) et commande (5V) via optocoupleurs. Protection par fusibles PTC 3A et câblage conforme (1,5 mm² pour 12V, 0,5 mm² pour signaux).

AS9100 (Traçabilité aéronautique) : Horodatage de chaque pesée, journalisation des missions avec ID unique, et versionnement des paramètres de calibration pour anticiper l'intégration en environnement certifié.

Choix technologiques justifiés

Raspberry Pi 5 : Performance +200% vs RPi4 (Cortex-A76), réduisant la latence de planification de 150 ms à 50 ms. Connectivité PCIe pour extensions futures.

Batterie LiPo 3S 11,1V : Garantit le couple moteur nominal (12V $\pm$ 10%).

Ponts H L298N : Supporte 2A continus vs 1A des DRV8833, nécessaire pour les 1,77A des moteurs RS PRO avec marge de sécurité 15%. Température de jonction <80°C maintenue.

Magasin ICAM :

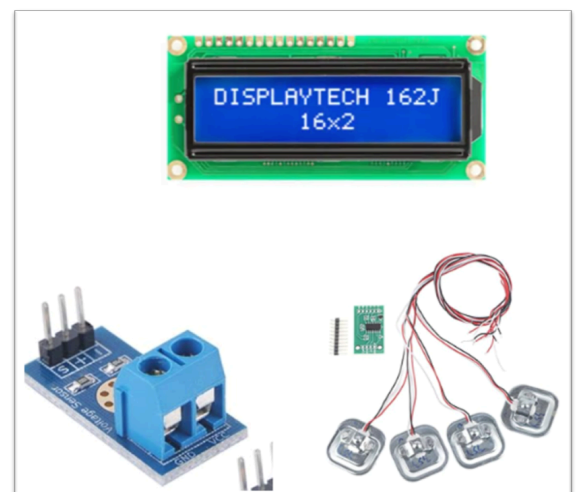
- Leds (rouges-vertes) + résistances
- Afficheur monochrome LCD
- Bouton d'arrêt d'urgence
- Interrupteur à bascule pour Fonctionnement général de l'AGV
- Carte Raspberry Pi 5
- Boitier Raspberry Pi 5 ventilo
- Carte Arduino MEGA
- Pont H x2
- Capteur EAI YDLIDAR

Partie cycle :

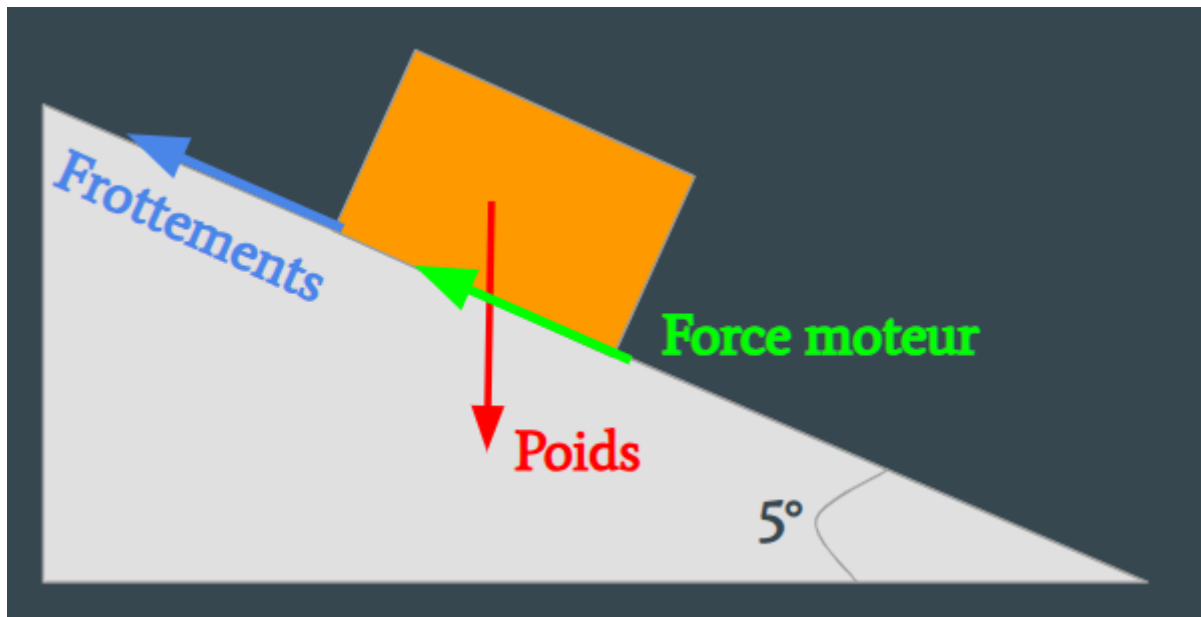
- Moteurs Motoréducteur CC RS PRO, 12 V c.c., 21.2W, dia. de l'arbre 6 mm
- Roues mecanum diamètre 80mm avec accouplement 6mm

A commander :

- Capteur de tension : [lien](#)
- cellule de peser : [lien](#) ou autre type (cf magasin)



Calcul - Dimensionnement des moteurs :



1. Le poids

$$P = m \cdot g = 9.81 \cdot 6 \cdot \sin(5) = 5.10 \text{ N}$$

2. Force de frottement

Force de frottement :

$$F_f = \mu \times m \times g = 0,050 \times 6,0 \times 9,81 = 2,94 \text{ N}$$

3. Force d'inertie :

$$F_i = m \times a = 6,0 \times 0,50 = 3,00 \text{ N}$$

4. Force totale à vaincre

$$F_{\text{total}} = P + F_f + F_i = 3,00 + 2,94 = 11.04 \text{ N}$$

5. Force par moteur (4 roues)

Force par moteur :

$$F_{\text{moteur}} = F_{\text{total}} / 4 = 11.04 / 4 = 2.76 \text{ N}$$

6. Couple par moteur

$$C_{\text{moteur}} = F_{\text{moteur}} \times r = 2.76 \times 0,040 = 0.1104 \text{ N}\cdot\text{m}$$

Soit : 1.126 kg·cm

Avec une marge (x2):

$$C_{requis} = 1.126 \times 2,0 = 2.252 \text{ kg}\cdot\text{cm}$$

Afin d'assurer les fonctions requises, nous choisirons les moteurs suivants : CC RS PRO, 12 V c.c., 21.2W.

4) Détails techniques

- Partie Hardware (Solution non retenue):

Raspberry Pi 4

- Performance inférieure pour traitement avancé
- Moins d'évolutivité pour fonctionnalités futures
- Connectivité réduite (USB 2.0, pas PCIe)

Arduino UNO

- Broches insuffisantes (14 digital vs 54 sur MEGA)
- PWM limité (6 sorties vs 15 sur MEGA)
- Mémoire insuffisante pour 4 PID moteurs
- Complexification du câblage nécessaire

Batterie LiPo 2S

- Tension insuffisante pour moteurs 12V (7.4V vs 12V)
- Performance moteurs réduite de 61%
- Risque de calage en mouvement Mecanum
- Couple insuffisant pour démonstration

- Partie Hardware (Solution retenue):

Raspberry Pi 5

- Calcule la navigation : "Va du point A au point B en évitant les obstacles"
- Traite les données complexes : caméra, capteurs avancés
- Gère ROS 2 : communication entre tous les programmes
- Communication réseau : contrôle à distance depuis ton PC
- Matrice de transformation pour les 4 roues
- Commandes individuelles pour chaque moteur
- Mouvements omnidirectionnels : latéraux, diagonaux, rotation

Boitier raspberry ventilo

- Pas de thermal throttling (ralentissement CPU)
- Performance stable même sous charge
- Longévité de la carte préservée

Arduino Méga

- Lit les capteurs en temps réel : encodeurs, télémètres, IMU
- Applique les PID : maintient la vitesse précise des moteurs
- Gère la sécurité : arrêt d'urgence si problème
- Communique avec la RPi5 : reçoit les ordres, envoie les données
- 15 sorties PWM → assez pour 4 moteurs + capteurs

Pont H L298N x2

- Amplifie le courant : 20mA Arduino → 2A pour les moteurs
- Gère la direction : avant/arrière/stop
- Contrôle la vitesse : PWM pour vitesse variable
- Protège les circuits : isole puissance et contrôle

Tableau récapitulatif des fonctions - Exigences - Test :

Fonctions	Exigences	Tests	Critère de succès
FP1 : Se déplacer de manière autonome à 0,5 m/s	Déplacement autonome sur 20 m avec déviation $<\pm 5$ cm, vitesse 0,5 m/s $\pm 0,05$ m/s.	Parcourir 20 m avec déviation $<\pm 5$ cm, 5 essais, vitesse mesurée.	Déviation $<\pm 5$ cm, vitesse dans plage, 95 % succès.
FP2 : Transporter une charge utile de 3 kg maximum	Transporter 3 kg sur 20 m sans basculement ($>5^\circ$), stabilité assurée.	Transporter 3 kg sur 20 m sans basculement, 5 cycles.	Stabilité maintenue, 100 % des cas.
FP3 : Contrôler la charge (nombre de pièces)	Déterminer le nombre de pièces à l'unité près (via balance ± 2 g).	Mesurer 3 kg (± 2 g) et compter pièces, 5 essais.	Précision ± 2 g, comptage exact, 100 %.

Fs1 : Communiquer avec un système externe	Taux de succès 98 % sur 50 transmissions, portée 20 m, délai <1 s, masse (± 2 g) et position.	50 transmissions WiFi, mesurer portée et délai.	98 % succès, portée 20 m, <1 s.
Fs2 : Retour automatique au poste de charge	Retour en <1 min à <20 % batterie, trajet <20 m, 5 succès consécutifs.	Retour mesuré <1 min à <20 % batterie, 5 essais.	Temps <1 min, 100 % succès.
Fs3 : Interface utilisateur locale	Affichage <2 s de poids (± 2 g), batterie (%), destination, erreurs, codes couleurs	Affichage vérifié <2 s par 5 utilisateurs.	Lisibilité validée, 100 % des cas.
Fs4 : Maintenance et diagnostic	Accès 80 % composants en <5 min, 5 codes d'erreur (ex. : E01) sur LCD.	Accès mesuré <5 min, 5 codes testés.	80 % accès, codes affichés, 100 %.
Fs5 : Ajustement dynamique selon charge	Vitesse de 0,5 m/s à 0,3 m/s avec 3 kg, ajustement <3 s après détection.	Test avec 3 kg, mesure vitesse et délai, 5 essais.	Ajustement <3 s, vitesse 0,3 m/s, 100 %.
Fs6 : Gestion énergétique	Autonomie 1 h à 3 kg, alerte LED clignotante à <20 % (± 5 %).	Test autonomie 1 h, alerte à <20 %, 3 cycles.	Autonomie >1 h, alerte fonctionnelle.
Fs7 : Gestion des exceptions	Détection <2 s, immobilisation <1 s, alerte LED rouge, affichage erreur <3 s.	Simulation (perte réseau, panne), 3 scénarios.	Toutes étapes <temps requis, 100 %.

Choix du châssis :

Châssis en profilés aluminium 20×20 mm

mm : Le châssis retenu utilise des profilés aluminium à section carrée 20×20 mm assemblés par équerres et visserie, formant une structure modulaire et rigide. Cette solution offre un excellent compromis entre légèreté, résistance mécanique et facilité d'assemblage pour un prototype évolutif. Le reste de l'ensemble sera habillé par du bois contreplaqué afin d'assurer les supports.



Alternatives écartées :

Châssis en tôle pliée acier : Bien que très robuste et économique en production série, l'acier alourdit significativement le véhicule (masse 2 à 3 fois supérieure à l'aluminium), réduisant l'autonomie et nécessitant des moteurs plus puissants. De plus, le pliage de tôle requiert un outillage spécialisé indisponible dans le contexte pédagogique du projet, rendant les modifications difficiles lors des phases de test et d'itération.

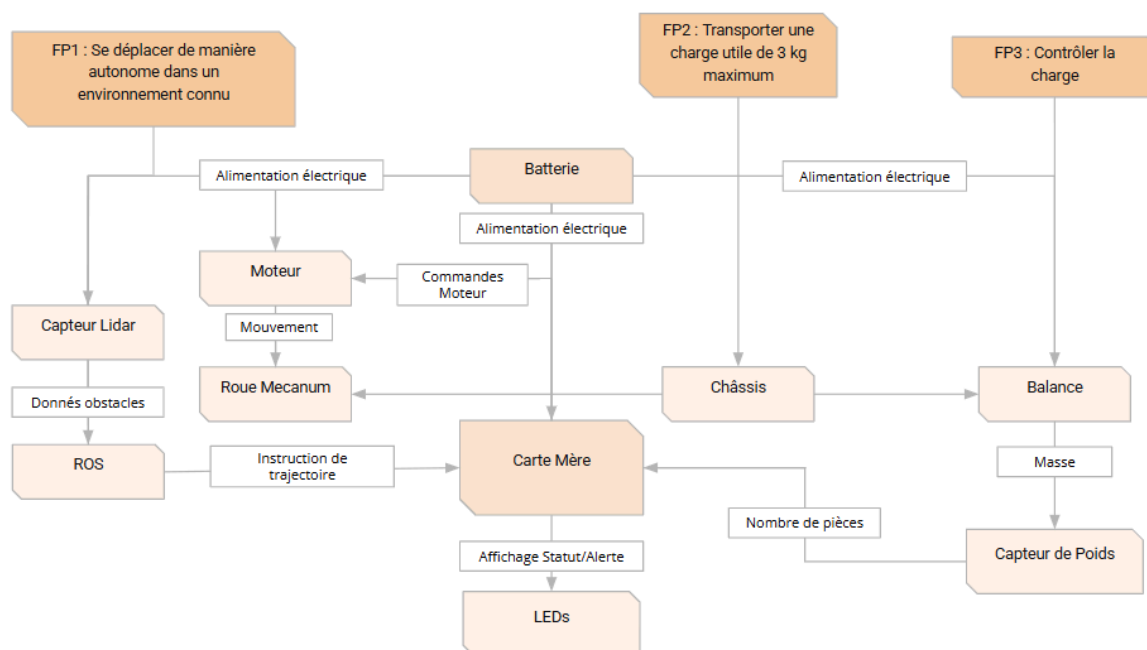
Châssis imprimé en PLA/ABS : L'impression 3D offre une grande liberté de conception et un coût matière faible, mais la résistance mécanique des plastiques est insuffisante pour supporter les charges dynamiques. Les zones de fixation des moteurs sont particulièrement critiques : les vibrations et les efforts de traction des roues Mecanum provoqueraient rapidement des fissures ou des déformations plastiques. De plus, le temps d'impression pour un châssis complet (estimé à 40-50 heures) et la fragilité des assemblages collés ou vissés dans le plastique rendent cette solution inadaptée à un usage intensif en tests.

Châssis en bois contreplaqué : Solution économique et facilement usinable avec les outils disponibles à l'ICAM (scie, perceuse), le contreplaqué présente néanmoins plusieurs inconvénients majeurs. Sa sensibilité à l'humidité et aux variations dimensionnelles compromet la précision de positionnement des composants critiques (LiDAR, moteurs). L'aspect visuel moins professionnel et la difficulté à obtenir des assemblages précis et répétables sans jeux excessifs pénalisent la qualité perçue du prototype final.

Justification de l'aluminium 20×20 : Les profilés aluminium combinent tous les avantages requis pour ce projet. Leur légèreté préserve l'autonomie énergétique tout en supportant sans déformation les 3 kg de charge utile plus la masse embarquée (Raspberry Pi, Arduino, batteries, moteurs). La modularité du système à rainures en T permet de positionner facilement les composants lors des ajustements de conception, sans perçage ni usinage supplémentaire. Les équerres standardisées garantissent des assemblages perpendiculaires précis, essentiels pour l'alignement des roues Mecanum et la stabilité de la plateforme de pesée. Enfin, l'esthétique industrielle des profilés anodisés renforce la crédibilité du prototype lors des démonstrations et correspond aux standards visuels des AGV commerciaux, facilitant la projection vers une éventuelle industrialisation.

Rappel du BDS effectué et qui est donc désormais incomplet avec l'évolution de notre travail et des solutions choisies.

BDS



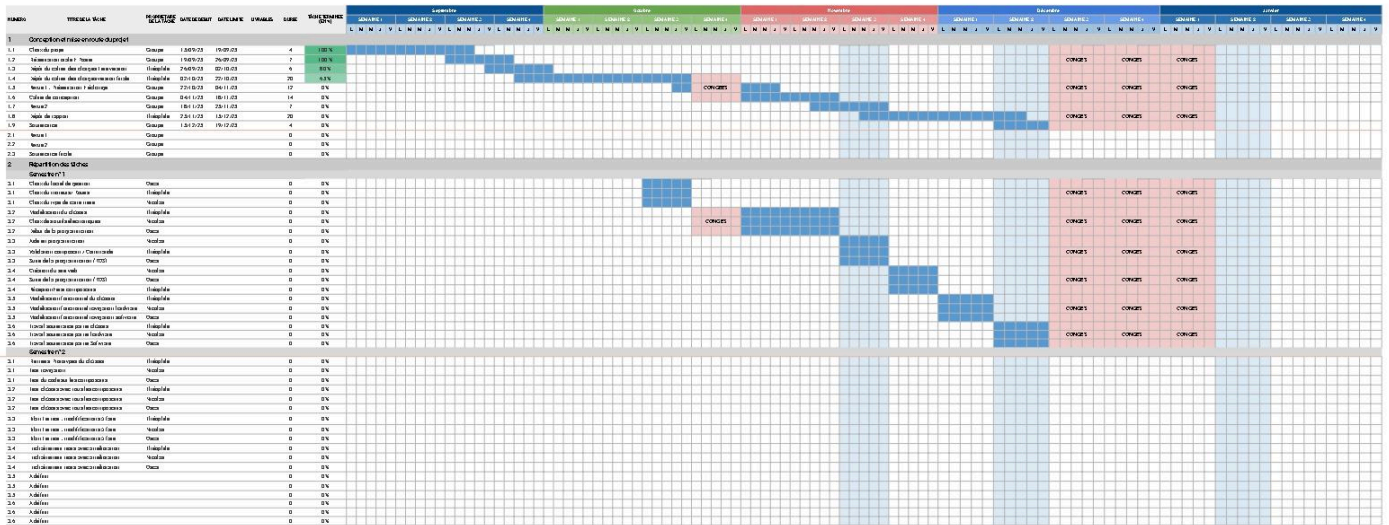
5) Planification et phases de développement

Étapes du développement

→ Diagramme de Gantt afin d'organiser l'avancé du projet tant sur le plan administratif que pratique

MODÈLE DE DIAGRAMME DE GANTT

* Visualisation du projet à jour - Répondre à vos demandes
 N° de projet : 101
 N° de version : 101
 Date : 10/10/2020



Test et validation

6) Annexe

Glossaire des termes techniques

AGV (Automated Guided Vehicle) : Véhicule autonome utilisé en logistique pour transporter des charges sans intervention humaine.

Arduino : Microcontrôleur open-source utilisé pour le contrôle temps réel des actionneurs et capteurs.

Boucle PID (Proportionnel, Intégral, Dérivé) : Algorithme de contrôle permettant d'asservir un système en corrigeant l'erreur entre la consigne et la mesure.

Capteur LiDAR (Light Detection and Ranging) : Capteur utilisant des lasers pour mesurer les distances et reconstituer l'environnement en 3D.

Cellule de charge : Capteur de force permettant de mesurer une masse via la déformation d'un corps d'épreuve.

Encodeur : Capteur monté sur un moteur permettant de mesurer sa vitesse ou sa position angulaire.

ERP (Enterprise Resource Planning) : Système d'information centralisé pour la gestion des processus d'une entreprise.

FP (Fonction Principale) : Fonction essentielle du système, directement liée à son objectif principal.

Fs (Fonction de service) : Fonction secondaire assurant le confort, la sécurité ou la maintenance.

HX711 : Amplificateur et convertisseur analogique-numérique spécialisé pour les jauges de contrainte.

Interface REST (Representational State Transfer) : Architecture de communication web utilisant des requêtes HTTP (GET, POST, PUT, etc.).

Mécanum (roues) : Roues équipées de rouleaux inclinés permettant un mouvement omnidirectionnel.

MES (Manufacturing Execution System) : Système de gestion et de suivi de la production en temps réel.

Odométrie : Technique d'estimation de la position d'un robot à partir de la rotation de ses roues.

PWM (Pulse Width Modulation) : Technique de modulation de signal permettant de contrôler la puissance délivrée à un actionneur.

Raspberry Pi : Nano-ordinateur utilisé comme cerveau décisionnel pour exécuter des algorithmes complexes.

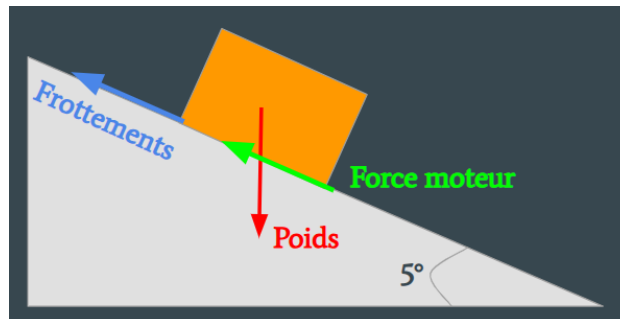
ROS (Robot Operating System) : Framework open-source pour le développement d'applications robotiques.

SLAM (Simultaneous Localization and Mapping) : Technique permettant à un robot de se localiser tout en cartographiant son environnement.

Topic ROS : Canal de communication asynchrone utilisé pour l'échange de messages entre nœuds ROS.

Annexe scientifique :

Mécanique :



1. Le poids

$$P = m * g * \sin(\alpha) = 6 * 9.81 * \sin(5) = 5.10 \text{ N}$$

2. Force de frottement

$$F_f = \mu * m * g = 0.050 * 6 * 9.81 = 2.94 \text{ N}$$

3. Force d'inertie :

$$F_i = m * a = 6 * 0.50 = 3.0 \text{ N}$$

4. Force totale à vaincre

$$F_{\text{totale}} = P + F_f + F_i = 3 + 2.94 + 5.10 = 11.04 \text{ N}$$

5. Force par moteur (4 roues)

$$F_{\text{moteur}} = \frac{F_{\text{totale}}}{4} = \frac{11.04}{4} = 2.76 \text{ N}$$

6. Couple par moteur

$$C_{\text{moteur}} = F_{\text{moteur}} * r = 2.76 * 0.040 = 0.1104 \text{ N.m}$$

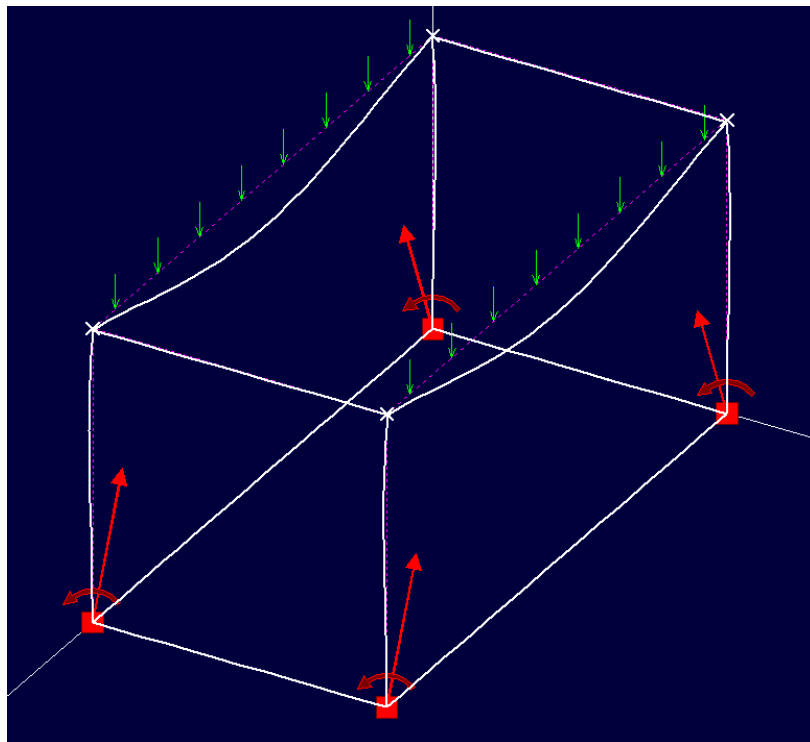
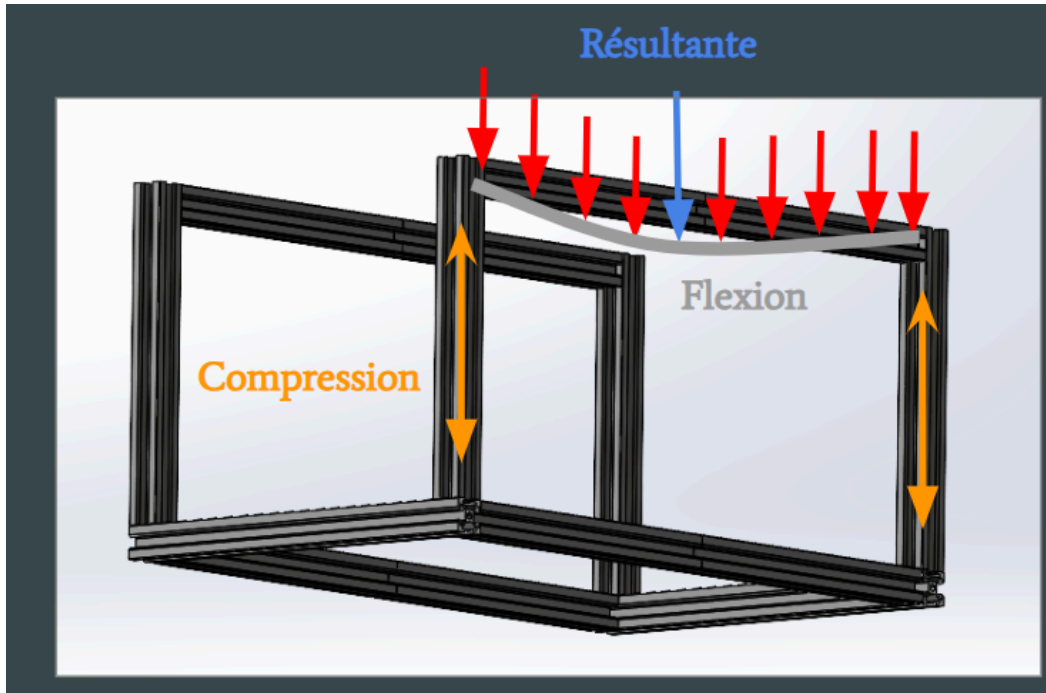
Soit : 1.126 kg.cm

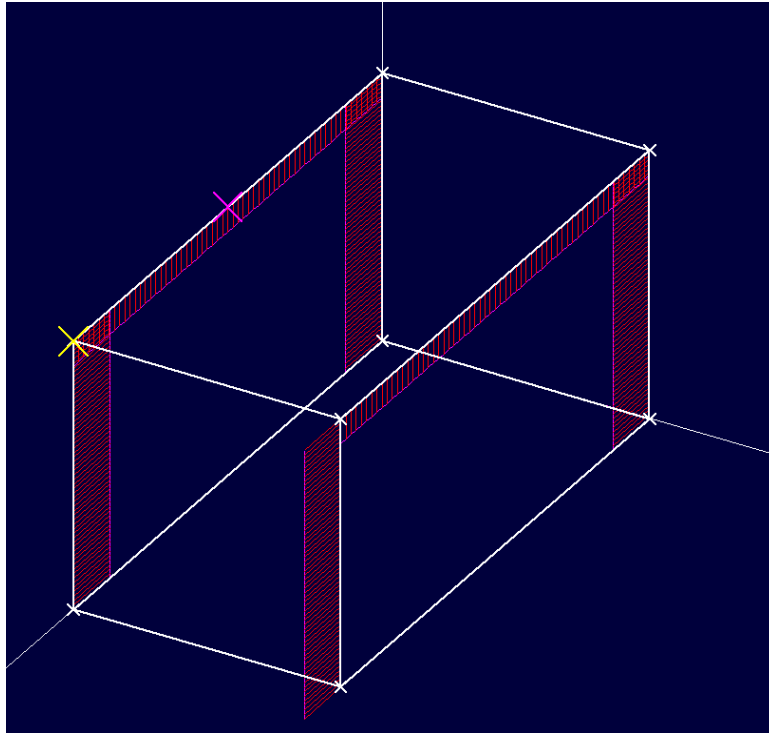
Avec une marge (x2):

$$C_{\text{requis}} = 1.126 * 2 = 2.252 \text{ Kg.cm}$$

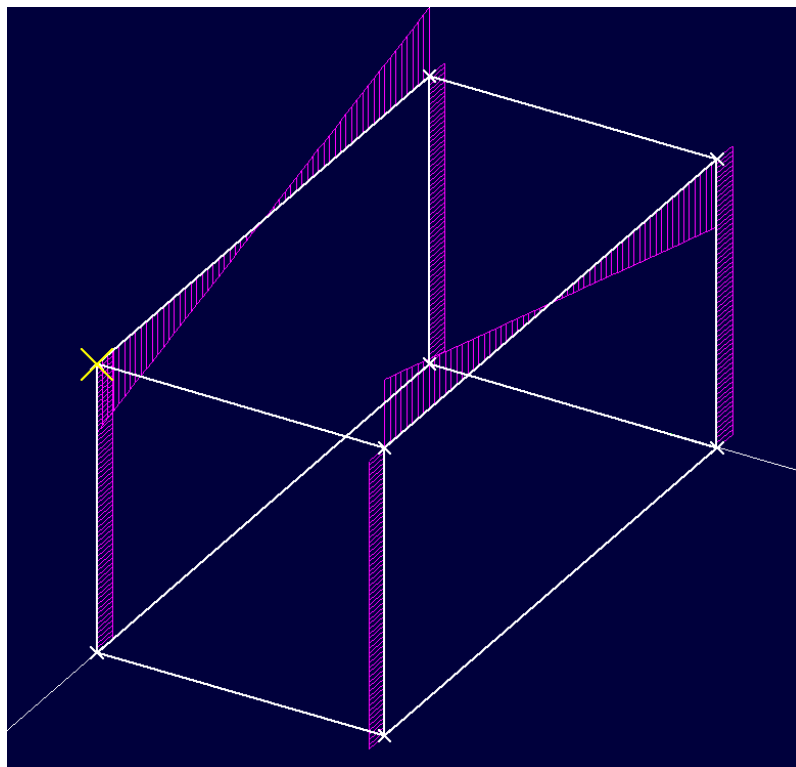
On retient donc un couple nécessaire de 2.252 Kg/cm pour chaque roue de notre AGV.

Modélisation :

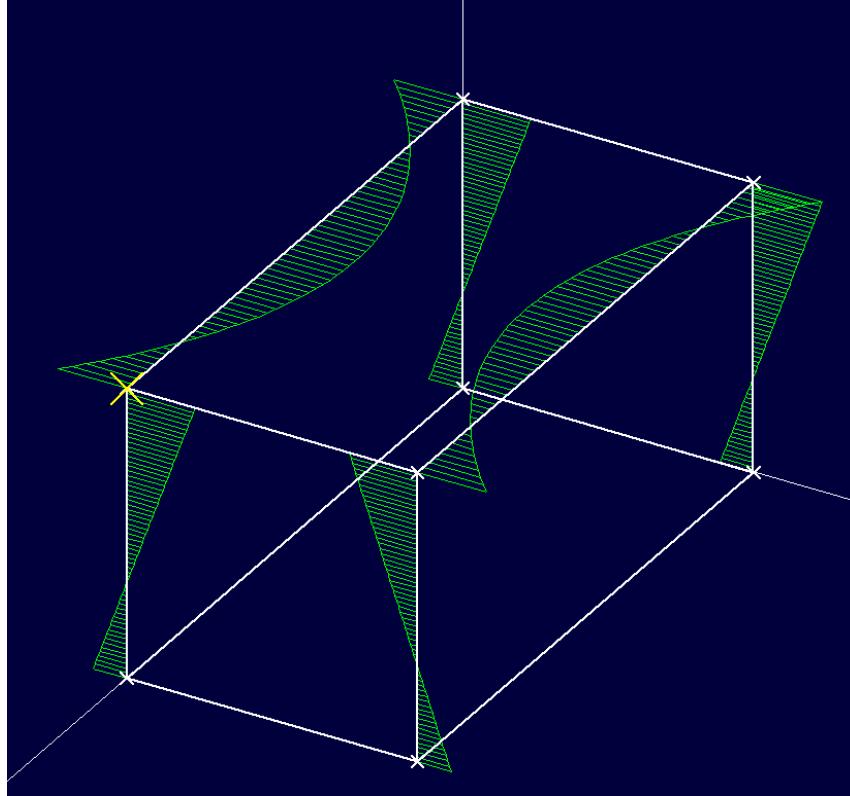




Modélisation des efforts normaux



Modélisation des efforts tranchants



Modélisation des moments fléchissants

Après analyse des résultats avec le logiciel RDM7, nous obtenons un fléchissement maximal de 0.06 mm ce qui est faible et donc parfait pour notre projet.

Calculons maintenant théoriquement la flèche située au centre de la poutre :

$$f_{\max} = \frac{5 q L^4}{384 E I}$$

Données :

- Profilé retenu : tube carré aluminium $40 \times 40 \times 2$ mm
- Moment d'inertie : $I = 18,43 \text{ cm}^4 = 18,43 \times 10^{-8} \text{ m}^4$ (cf. datasheet)
- Module d'Young Aluminium : $E = 69\,000 \text{ MPa} = 69 \times 10^9 \text{ Pa}$

Calcul détaillé :

$$f_{\max} = \frac{5 \times 171,68 \times (0,4)^4}{384 \times 69 \times 10^9 \times 18,43 \times 10^{-8}}$$

$$f_{\max} = \frac{5 \times 171,68 \times 0,0256}{384 \times 69 \times 10^9 \times 18,43 \times 10^{-8}} = \frac{21,98}{48\,945\,216} = 4,49 \times 10^{-7} \text{ m}$$

La flèche maximale théorique est donc de 0.045 mm.

La différence avec le résultat de la modélisation peut s'expliquer par le fait qu'on ait approximé la section dans la simulation à un tube carré, donc moins résistant que la section réelle, avec une forme complexe.

Dans les 2 cas, le fléchissement reste très faible et donc intéressant pour noter AGV. La structure aluminium est le bon choix pour le châssis de notre AGV.

Dimensionnement de l'alimentation:

On va fonctionner par pack

Premier pack P1 = 12V-2A :

- 2 x Pont H L298N
- 4 x Moteur DC encodeur 12V-2A

Deuxième pack :

$$P_{\text{entrée}} = P_{\text{sortie}} / \text{efficacité} = 35\text{W} / 0.9 = 38.89\text{W} \sim 40\text{W}$$

- Convertisseur (buck) 90%

- Raspberry Pi 5V-5A
- Capteur LIDAR YDLIDAR x4 pro 5V-2A
- IMU 5V-3mA

Troisième pack P3 = 12V-0.3A :

- Arduino MEGA 12V-0.3A
- LCD + LED 5V-0.2A
- Cellule de pesée 5V-0.2A

$$P1 = 4 \times (12V \times 2A) = 96W$$

$$P2 = 38.9W$$

$$P3 = 12V \times 0.3A = 3.6W$$

$$P_{tot} = P1 + P2 + P3 \sim 150W$$

Si batterie LiPo 3S 11.1 V nominal (12.6V chargé) :

Pour 1h d'autonomie :

- Intensité moyenne : $I_m = 150W/12V = 12,5A \sim 15A$
- Capacité minimal : $15 \times 1h = 15 Ah$
- Capacité réelle (LiPo à 80%) : $15A/0.8 = 18750 mAh$

Pour 45min d'autonomie (largement suffisant) :

- Intensité moyenne : $I_m = 150A/12V = 12A$
- Capacité minimal : $15A \times 0.75h = 11.25 Ah$
- Capacité réelle (LiPo à 80%) : $11.25Ah/0.8 = 14.062 Ah$

Calcul du taux de décharge :

(Pour batterie 7Ah)

$$C_{min} = 15A/7Ah = 2.14C$$

$$C_{pic} (\text{pic moteurs}) \approx 20-25A/7Ah = 1.3-1.7C$$

Schéma électronique des composants pour le projet :

